

IAM Request Demand and Capacity Optimization

A code-aligned probabilistic model for IAM request workload

Demand	Workload	Decision
request count	request-level weights	protected capacity

This report documents the current portfolio prototype. It follows the implementation path from `RequestObservation` to the final capacity plan and reports the results of the reproducible synthetic experiments. It does not claim production accuracy or vendor-specific behavior.

1 Problem and modeling objective

IAM capacity is often planned from historical averages and operational experience. That approach can hide two distinct uncertainties: how many requests will arrive, and how much active backend capacity each request will consume. The model makes both quantities explicit.

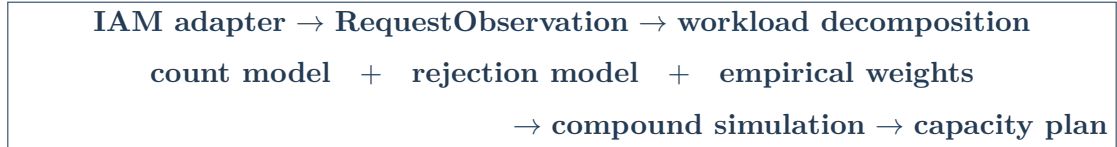
For a future planning bucket t , the target is a predictive distribution for the total request workload W_t , not only a point estimate:

$$W_t = \sum_{i=1}^{N_t} w_{i,t}$$

Here N_t is the future request count and $w_{i,t}$ is the active workload of request i . The capacity policy then protects a selected quantile of W_t . The current prototype uses P95 for the main experiment.

2 Implementation flow

The code is deliberately split into small contracts. Data acquisition remains outside the model, so a future IAM adapter can map vendor records into the same request observation interface.



2.1 Request workload contract

The request-level workload is implemented in `request_contract.py`. It separates work that happens before the outcome, approval-workflow execution, and backend tasks that are conditional on approval:

$$w_i = w_{i,\text{pre}} + w_{i,\text{workflow}} + \mathbb{I}_{\{\text{approved}\}} w_{i,\text{backend}}$$

Backend workload is the sum of resource-time across observed tasks:

$$w_{i,\text{backend}} = \sum_j \text{active_seconds}_{ij} \text{capacity_units}_{ij}$$

Workflow workload is the active execution used to calculate approval steps, resolve approvers, and route the request:

$$w_{i,\text{workflow}} = \text{workflow_overhead_seconds}_i \text{workflow_capacity_units}_i$$

This sum preserves capacity consumed by parallel tasks. A rejected request still pays the pre-work and workflow costs; only post-approval backend tasks are gated by the outcome. Human waiting time is not counted as active capacity unless the system actually holds a capacity unit while waiting.

2.2 Historical aggregation and empirical distributions

`workload.py` aggregates observations into period-level request counts and total workloads. It then fits one weighted empirical distribution for approved requests and one for rejected requests. No Normal assumption is imposed on request workload values.

The observations remain visible as actual workload values. Their influence is controlled by a recency weight:

$$a_j = \lambda^j, \quad 0 < \lambda \leq 1$$

Here $j = 0$ is the newest observation. With the experiment's $\lambda = 0.995$, old observations are gradually down-weighted without being discarded. The weighted empirical sampler preserves skew and high-cost tails.

2.3 Request-count forecast

`forecast.py` fits a Negative Binomial baseline to historical period counts using method-of-moments estimates. If μ is the sample mean and v the sample variance, the model uses:

$$v = \mu + \phi\mu^2, \quad \hat{\phi} = \max\left(0, \frac{v - \mu}{\mu^2}\right)$$

The ϕ term is the dispersion parameter. When observed variance is larger than the mean, it allows more variation than a Poisson model. Future counts are sampled through the equivalent Gamma–Poisson mixture: first sample a Poisson rate from a Gamma distribution, then sample the request count from that rate.

The class also exposes an online correction state:

$$s_{t+1} = \omega + \rho s_t + \eta \text{score}_t$$

The current experiment uses the static fit at each forecast origin; the update method is the interface for a later continuously updated deployment loop.

2.4 Rejection model and compound simulation

The rejection component estimates the probability that a request is rejected before backend activation. It stores the probability on the log-odds scale, which keeps the transformed forecast between zero and one. In each simulation, the flow samples a future request count, samples an outcome for every request, samples a workload from the matching empirical distribution, and adds all request workloads into one simulated W_t .

2.5 Capacity decision

The resulting simulated sample is passed to `capacity.py`:

$$C_{\text{requests},t} = Q_{0.95}(W_t), \quad C_{\text{spare},t} = \max(0, C_{\text{total},t} - C_{\text{requests},t})$$

If protected request capacity exceeds total capacity, the difference is reported as a deficit. This makes request priority explicit instead of hiding it inside an average utilization number.

3 Implementation map

Module	Responsibilities
<code>request_contract.py</code>	Request observations, backend tasks, workload contract
<code>workload.py</code>	Aggregation, recency weights, empirical fit and sampling
<code>forecast.py</code>	Count model, rejection model, compound simulation
<code>capacity.py</code>	Quantile protection, spare capacity, deficit
<code>flow.py</code>	Parameter validation and end-to-end orchestration
<code>examples/</code>	Synthetic data and reproducible experiment runners

4 Experimental design

The main experiment is synthetic so that the complete flow can be reproduced without exposing real IAM data. It contains 120 daily periods, approximately 200 requests per period, an approximately 11% rejection rate, overdispersed counts, workflow execution for every request, and an 8% high-cost backend tail among approved requests.

The capacity experiment uses 10,000 Monte Carlo simulations, a P95 protection target, total capacity of 3,000 capacity-unit seconds per period, and seed 20260920. The separate walk-forward evaluation uses 84 training periods, 36 held-out periods, and 1,000 simulations at each evaluation origin.

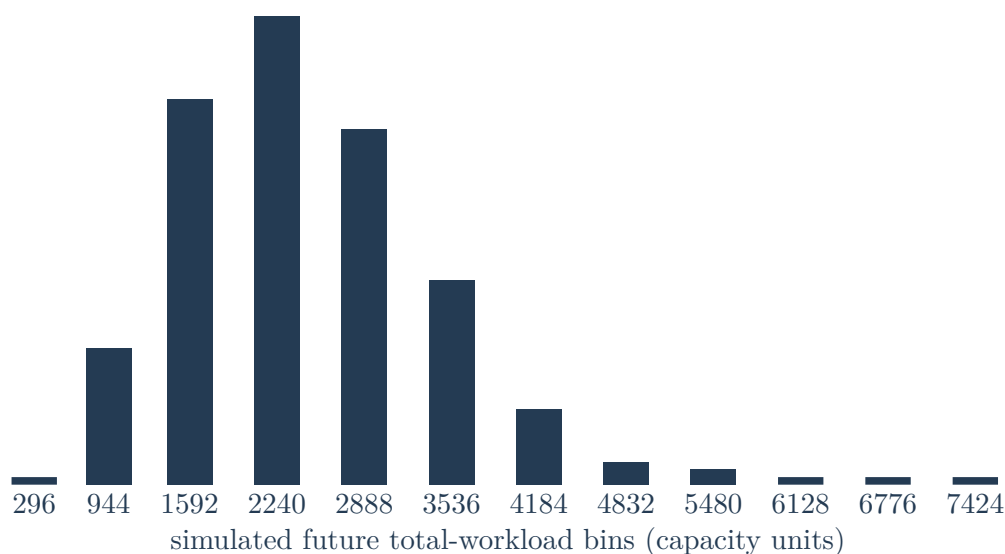
5 Results

5.1 Main capacity experiment

The generated dataset contains 23,431 request observations. The observed rejection rate is 10.97%. The fitted request-count mean is 195.26 requests per period, and the approved/rejected workload means are 15.47 and 2.70 capacity units per request respectively.

Quantity	Value
Simulated total-workload P50	2,671.88
Simulated total-workload P75	3,314.21
Simulated total-workload P90	3,981.97
Simulated total-workload P95	4,391.47
Simulated total-workload P99	5,431.87
Configured total capacity	3,000.00
Protected request capacity	4,391.47
Reported capacity deficit	1,391.47

The compact histogram below is a visual summary of the simulated workload distribution. Bars are scaled relative to the largest bin.



5.2 Walk-forward evaluation

At each origin, the model sees only the preceding periods, forecasts the next period, and compares the result with the held-out request count and workload.

Metric	Result
Request-count MAE	60.88 requests
Request-count P95 coverage	91.67%
Workload P50 MAE	842.99 capacity units
Workload P95 coverage	91.67%
Average-weight MAE	0.52 units/request
Average predicted rejection rate	11.07%
Average actual rejection rate	10.59%

The weight estimate is relatively stable in this synthetic scenario, while the request count is harder to predict. Since total workload is a compound sum, count uncertainty propagates directly into workload uncertainty.

5.3 Count-model scenario comparison

The isolated scenario experiment compares the stationary baseline with a seasonal extension. The result is diagnostic rather than a model-selection claim:

Scenario	Model	MAE	P95 coverage
Stationary	Stationary NB	61.28	94.44%
Stationary	Seasonal NB	60.02	94.44%
Weak seasonal	Stationary NB	70.84	83.33%
Weak seasonal	Seasonal NB	72.00	80.56%
Strong seasonal	Stationary NB	49.81	100.00%
Strong seasonal	Seasonal NB	38.67	100.00%
Random shocks	Stationary NB	83.74	94.44%
Random shocks	Seasonal NB	82.50	91.67%

The comparison illustrates why the current Negative Binomial component should be treated as a baseline: it can represent overdispersion, but it does not automatically learn seasonality or external drivers.

6 Interpretation and limitations

The portfolio result is not that the synthetic model is already production- accurate. It is that the capacity question has been decomposed into auditable components with an explicit risk policy:

- request demand is modeled separately from request cost;
- request cost is derived from active workflow and backend work;
- approval outcomes determine whether backend workload is activated;
- simulation converts component uncertainty into a total-workload distribution;
- the P95 quantile turns that distribution into a request-priority capacity decision.

The current results are limited by the synthetic data, the pooled count fit, the method-of-moments baseline, and the absence of a real queue/service-level model. A production-oriented next step would use aggregated and anonymized IAM observations, establish a rolling backtest protocol, and add temporal covariates or an online likelihood-based update only where the data supports it.

7 Reproducibility

From the repository root:

```
PYTHONPATH=src python3 examples/run_capacity_experiment.py
PYTHONPATH=src python3 examples/run_walk_forward_evaluation.py
PYTHONPATH=src python3 examples/run_count_model_scenarios.py
```

The numbers in this report correspond to the seeds and configurations stated in Section 5. Generated CSV/JSON outputs are intentionally not part of the portfolio repository; the scripts regenerate them when needed.